

4x4 Accelerator

v1.0 Final Specification

Signed INT8 matrix multiplication accelerator based on a 4x4 systolic PE array, input skewing, operand buffering, and an FSM controller.

Item	v1.0 Definition
Operation	$C = A \times B$, fixed 4x4 matrices
Input format	Signed INT8
Accumulator/output	Signed 32-bit
Compute array	16 processing elements (4 x 4)
External control	load, start, busy, done
Verification	Directed unit tests + Python golden-model vector-driven end-to-end test

1. Scope and Goals

Version 1.0 implements a complete fixed-size 4x4 signed integer matrix multiplier. The design accepts two 4x4 signed INT8 matrices, stores them, sequences four k-slices through a systolic datapath, accumulates the 16 output elements, and reports completion through a one-cycle done indication.

The v1.0 goal is functional RTL architecture and verification clarity. It intentionally uses register-based operand storage rather than SRAM and does not implement ready/valid command handshakes, ping-pong buffering, variable matrix dimensions, saturation, or UVM.

2. Top-Level Interface

```
module accelerator_4x4_top #(
    parameter int DATA_W = 8,
    parameter int ACC_W = 32
) (
    input logic clk,
    input logic rst_n,
    input logic load,
    input logic start,
    input logic signed [DATA_W-1:0] a_matrix [0:3][0:3],
    input logic signed [DATA_W-1:0] b_matrix [0:3][0:3],
    output logic busy,
    output logic done,
    output wire signed [ACC_W-1:0] c_out [0:3][0:3]
);
```

Signal	Dir.	Meaning
clk	In	System clock
rst_n	In	Asynchronous active-low reset
load	In	Capture both input matrices on a rising edge
start	In	Request one matrix-multiply operation
a_matrix	In	4x4 signed INT8 matrix A
b_matrix	In	4x4 signed INT8 matrix B
busy	Out	High during CLEAR, FEED, and DRAIN
done	Out	One-cycle completion indication
c_out	Out	4x4 signed ACC_W result matrix

2.1 External Protocol

The intended v1.0 sequence is: drive A and B, assert load for one rising edge, deassert load, pulse start, wait while busy is high, and consume c_out when done is asserted. The environment must not assert load while an operation is busy. A new start is issued only after loading is complete.

3. Architecture

```
accelerator_4x4_top
|
+-- accelerator_4x4_controller
|
+-- matrix_operand_buffer
|
+-- accelerator_4x4_core
```

```

|
+-- input_skew
|
+-- accelerator_4x4
|
+-- 16 x systolic_pe

```

The top module is structural integration. The controller owns operation sequencing; the operand buffer owns matrix storage and k-based selection; the core owns input skew and the 4x4 PE array. No duplicate FSM, skew stage, matrix arithmetic, or additional feed pipeline stage is present in the top.

4. Processing Element (PE)

Each PE forwards A one position horizontally and B one position vertically through registered outputs. When both input valid signals are high, it accumulates the signed product into its local ACC_W accumulator.

```

product = a_in * b_in;

if (!rst_n)
    acc <= 0;
else if (clear_acc)
    acc <= 0;
else if (a_valid_in && b_valid_in)
    acc <= acc + sign_extend(product);

```

For DATA_W=8 the full signed product width is 16 bits. The product is sign-extended to ACC_W before accumulation. clear_acc has priority over MAC accumulation.

5. 4x4 Systolic Interconnect

The array contains 16 PEs. A values propagate from left to right across each row; B values propagate from top to bottom down each column. Data and valid signals are forwarded together. Each PE therefore receives the appropriate A(i,k) and B(k,j) wavefront and accumulates $A(i,k) \cdot B(k,j)$.

```

A ---> PE(i,0) ---> PE(i,1) ---> PE(i,2) ---> PE(i,3)
      |           |           |           |
      v B         v B         v B         v B

```

$C(i,j)$ is held in the local accumulator of $PE(i,j)$.

6. Input Skew

Matrix rows/columns are presented together at the core boundary, so input_skew delays lane N by N cycles for $N=0..3$. Lane 0 is a combinational bypass; lanes 1, 2, and 3 contain exactly 1, 2, and 3 register stages respectively. Data and valid bits are delayed identically.

Lane	Delay
0	0 cycles
1	1 cycle
2	2 cycles
3	3 cycles

7. Matrix Operand Buffer

The operand buffer stores one complete A matrix and one complete B matrix in register arrays. load captures both matrices on a rising edge. With load low, stored operands retain their values.

```

a_feed[lane] = a_mem[lane][k_counter];
b_feed[lane] = b_mem[k_counter][lane];

```

Thus `k_counter` acts as a simple read address. For each FEED cycle, the buffer presents one A column `A[:,k]` and one B row `B[k,:]`. Feed selection is combinational and adds no pipeline cycle.

8. Controller

The controller is a Moore FSM with states IDLE, CLEAR, FEED, DRAIN, and DONE. start is accepted from IDLE. start pulses while already busy are ignored by the current state transitions.

State	busy	done	clear_acc	feed_valid	Duration / transition
IDLE	0	0	0	0	Wait for start
CLEAR	1	0	1	0	1 cycle
FEED	1	0	0	1	4 cycles, k=0..3
DRAIN	1	0	0	0	6 cycles
DONE	0	1	0	0	1 cycle, then IDLE

8.1 FEED Sequence

```

FEED cycle 1: k=0 -> A[:,0], B[0,:]
FEED cycle 2: k=1 -> A[:,1], B[1,:]
FEED cycle 3: k=2 -> A[:,2], B[2,:]
FEED cycle 4: k=3 -> A[:,3], B[3,:]

```

`feed_valid` is fanned out to all four A valid lanes and all four B valid lanes. After the four feed cycles, no new operands enter the core; the existing skewed/forwarded wavefront continues through the array during DRAIN.

9. Operation Timing

Phase	Cycles	Externally visible behavior
LOAD	1 rising edge	External protocol; captures A/B before start
CLEAR	1	busy=1, accumulators cleared
FEED	4	busy=1, k=0..3 operands injected
DRAIN	6	busy=1, internal wavefront completes
DONE	1	busy=0, done=1, completed c_out available

After start is accepted, busy covers 11 controller cycles: 1 CLEAR + 4 FEED + 6 DRAIN. DONE is then asserted for one cycle. `c_out` remains stored in the PE accumulators during idle until the next CLEAR or reset.

10. Arithmetic and Data Types

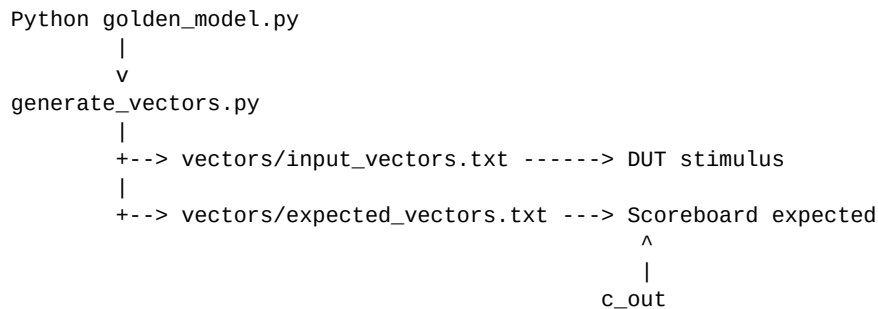
Inputs are signed `DATA_W` values; v1.0 uses `DATA_W=8`. Each PE forms a full $2 \times \text{DATA_W}$ signed product and sign-extends it into `ACC_W` before addition. v1.0 uses `ACC_W=32`. There is no saturation or explicit overflow handling beyond the configured accumulator width.

For the fixed 4-term INT8 dot product used by each C element, `ACC_W=32` is comfortably wider than the required mathematical range.

11. Reset Behavior

rst_n is asynchronous and active low. PE forwarding registers and accumulators, input-skew registers, controller state/counters, and operand-buffer storage are reset to zero/idle as implemented by their respective modules. The top-level end-to-end test verifies busy=0, done=0, and all c_out elements zero immediately after asynchronous reset.

12. Verification Architecture



The Python golden model performs functional signed INT8 4x4 matrix multiplication using Python integer accumulation. Deterministic vectors include identity/sequential data, signed values, zeros, and INT8 boundary values.

The final SystemVerilog top-level testbench does not recalculate $C=A*B$. It reads the Python-generated expected matrix and compares all 16 signed DUT outputs when done asserts. Normal vector cases run without global reset between operations, thereby exercising controller-driven clear_acc.

12.1 Verified Areas

Area	Status
PE MAC, forwarding, valid gating, reset, clear	PASS
4x4 horizontal/vertical interconnect	PASS
Input skew 0/1/2/3 cycles	PASS
Core 4x4 matrix multiplication	PASS
Python golden model sanity cases	PASS
Vector generation and file consistency	PASS
Controller CLEAR/FEED/DRAIN/DONE timing	PASS
Operand buffer load/hold/reload/k selection	PASS
Top-level load/start/busy/done integration	PASS
Vector-driven end-to-end scoreboard	PASS
Multiple operations without reset	PASS
Asynchronous reset after completed operation	PASS

13. Project File Structure

```

4x4_accelerator/
|
+-- rtl/
|   +-- systolic_pe.sv
|   +-- accelerator_4x4.sv
|   +-- input_skew.sv
|   +-- accelerator_4x4_core.sv
|   +-- accelerator_4x4_controller.sv

```

```
|   +-- matrix_operand_buffer.sv
|   +-- accelerator_4x4_top.sv
|
+-- tb/
|   +-- unit and integration testbenches
|   +-- tb_accelerator_4x4_top.sv
|
+-- model/
|   +-- golden_model.py
|   +-- generate_vectors.py
|
+-- vectors/
|   +-- input_vectors.txt
|   +-- expected_vectors.txt
|
+-- docs/
    +-- 4x4_accelerator_v1.0_specification.pdf
```

14. v1.0 Limitations and Future Extensions

v1.0 is deliberately compact. Potential later extensions include ready/valid command interfaces, protection against load while busy, SRAM-backed operand storage, configurable matrix dimensions, larger arrays, tiling, ping-pong buffers, explicit overflow/saturation policy, performance counters, and a UVM verification environment. These are not requirements of the frozen v1.0 implementation.

15. v1.0 Completion Criteria

v1.0 is considered functionally complete when all implemented unit/integration tests and the final Python-vector-driven top-level end-to-end simulation pass. Based on the completed project workflow, those tests have passed and the v1.0 RTL is ready to be frozen for documentation and release.